

UAH Mars Drone ASW Architecture Design Document

Index

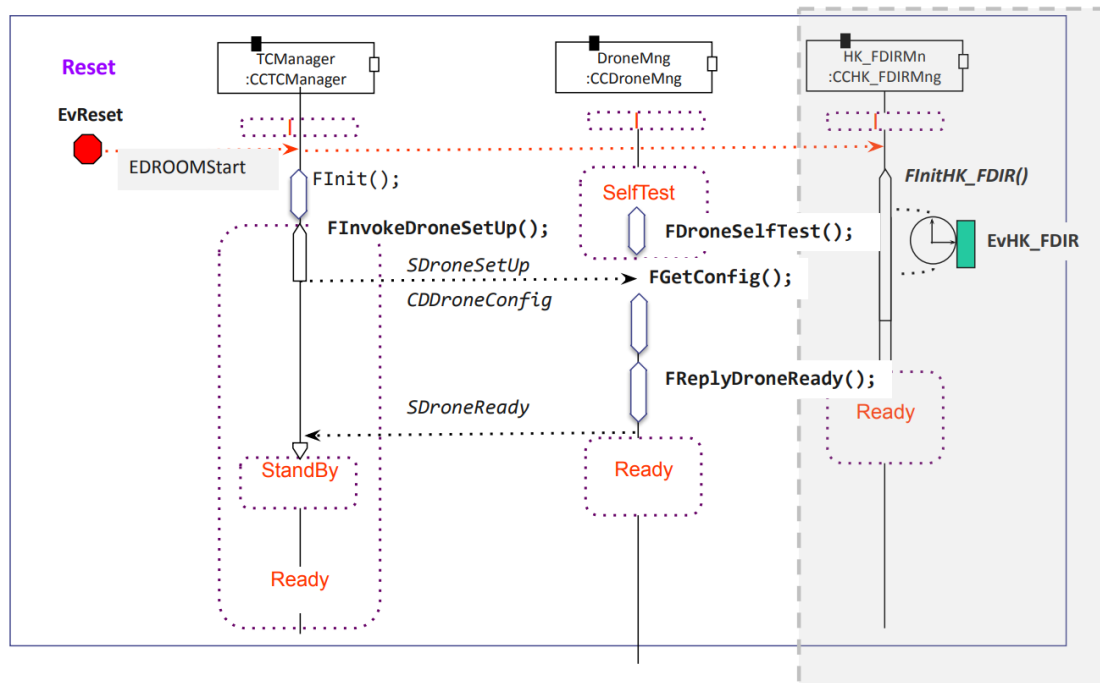
1. Introduction
2. Applicable and Reference Documents
3. Terms, definitions and abbreviated terms
4. Software Design Overview
5. Software Design
 - 5.1. Software Static Architecture
 - 5.2. Software Dynamic Architecture
 - 5.2.1. Computational Model
 - 5.2.2. Software Behaviour
 - 5.3. Real-Time Requirements
 - 5.4. Software Components Design
 - 5.5. SRD Req Coverage Matrix
6. Traceability

5.2.2 Software Behaviour

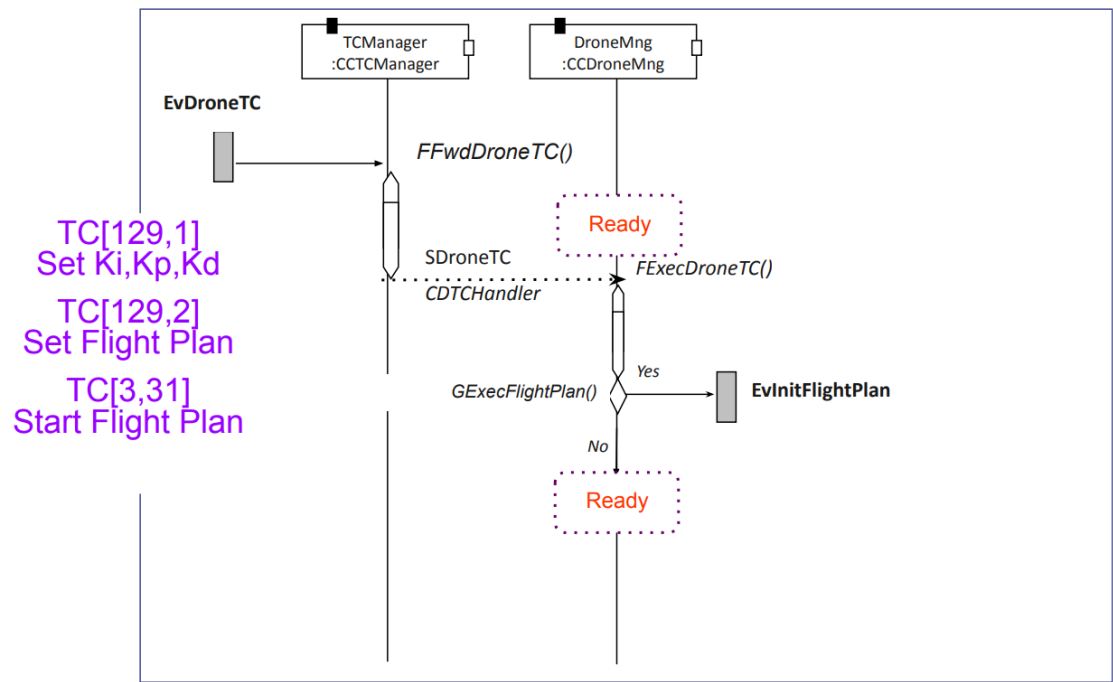
Scenarios

Event	Description
EvRxTC (external IRQ)	TC reception
EvAcceptedTC (internal)	TC acceptance
EvRejectedTC (internal)	TC rejection
EvToReboot (internal)	System Reboot
EvHK_FDIR (timing)	Periodic HK and FDIR Mng
EvHK_FDIR_TC (internal)	HK_FDIR TC Execution
EvReset (System)	System Reset and Initialization
EvDroneTC (internal)	Forwarding and distribution of Service 129 TCs to the drone
EvInitFlightPlan (internal)	Drone internal transition to start periodic guidance and navigation tasks
EvFlightCtrl (timing)	Expiration of the periodic 100ms timer governing the flight PID control loop
EvFlightPlanDone (internal)	Notification that the drone has successfully reached its target and landed

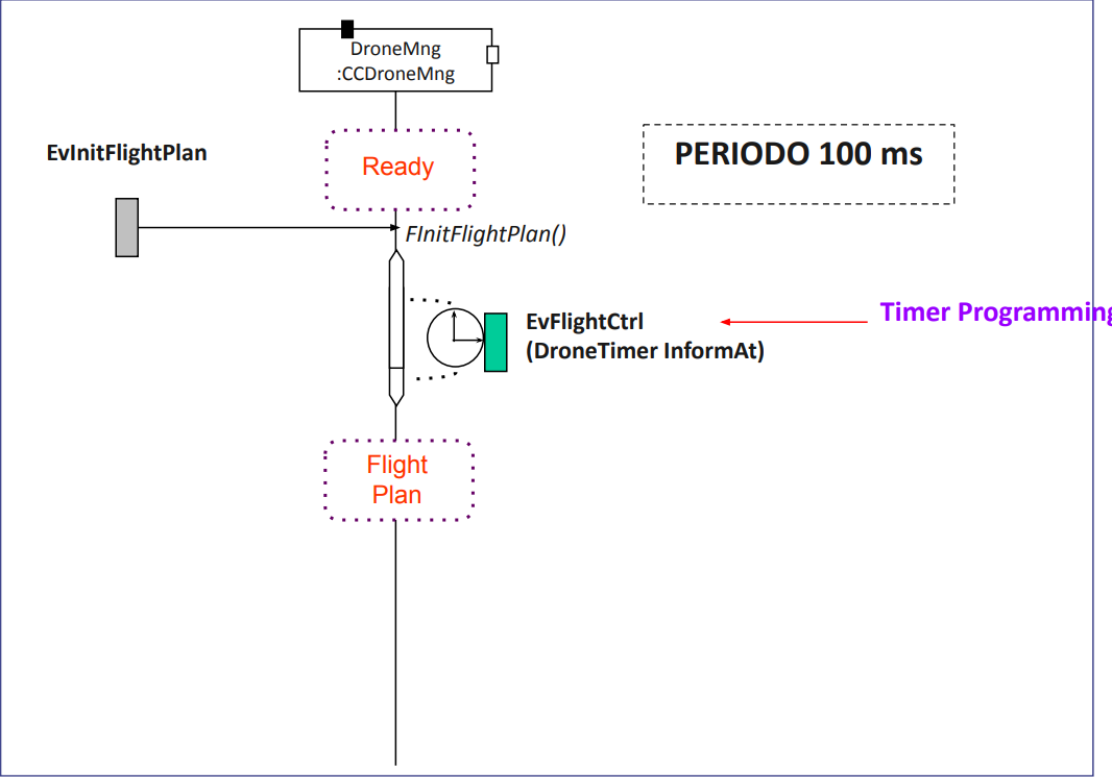
EvReset



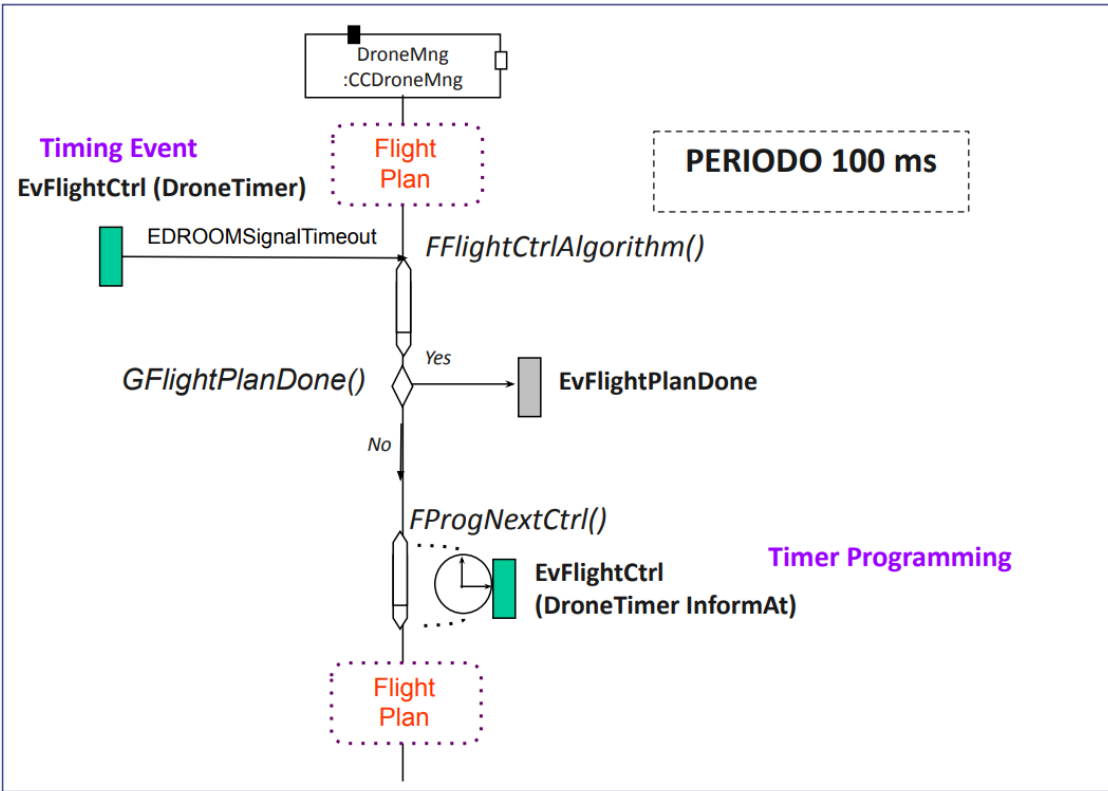
EvDroneTC



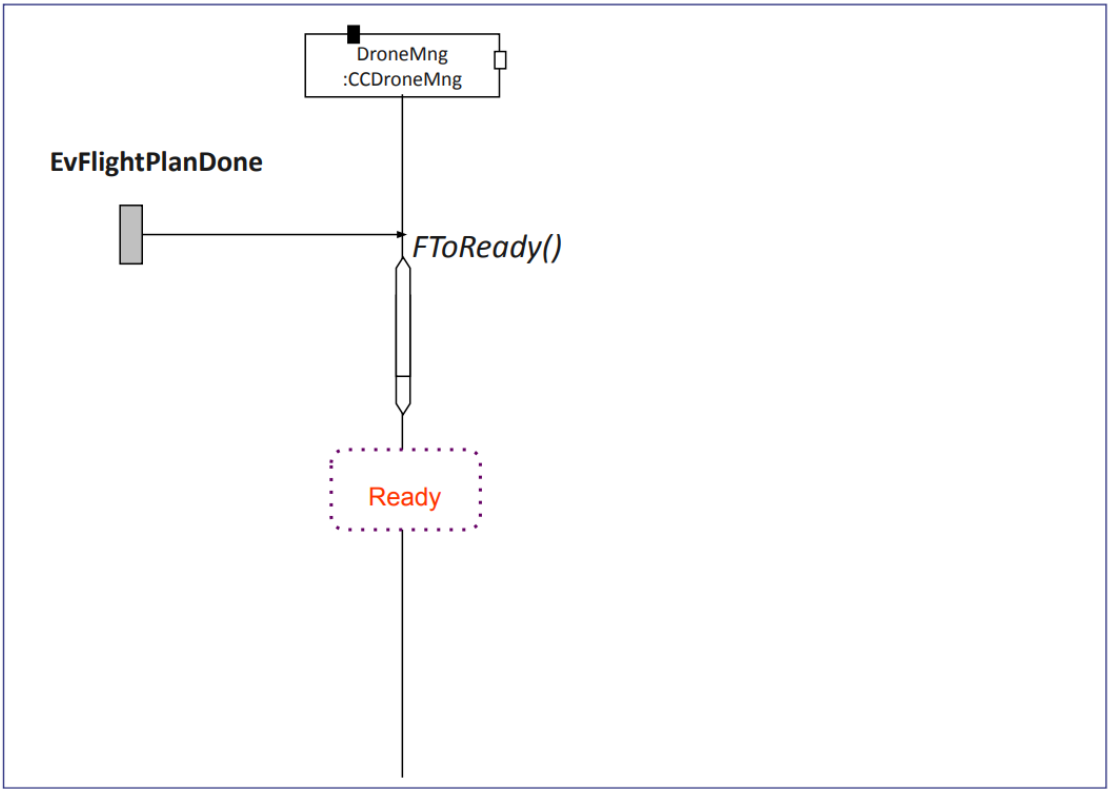
EvInitFlightPlan



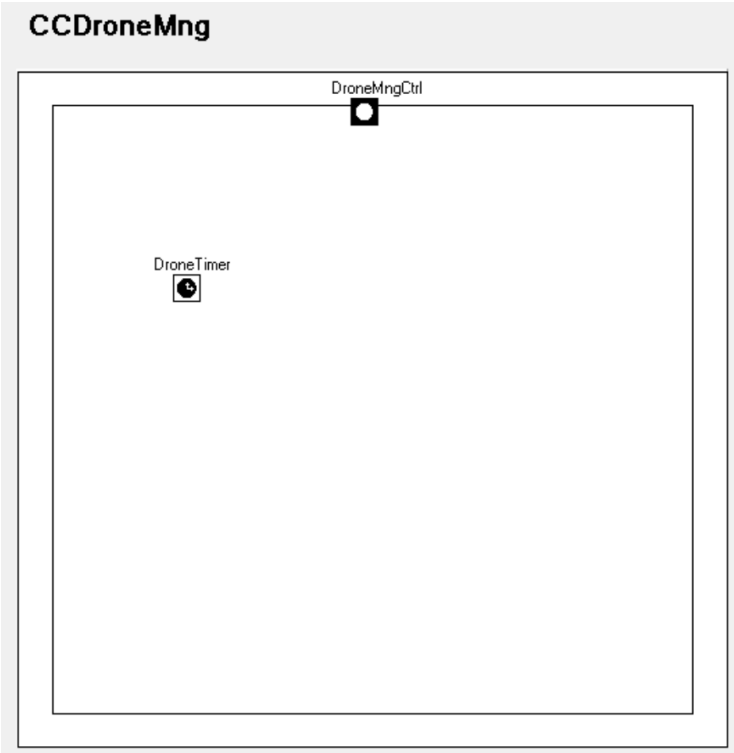
EvFlightCtrl



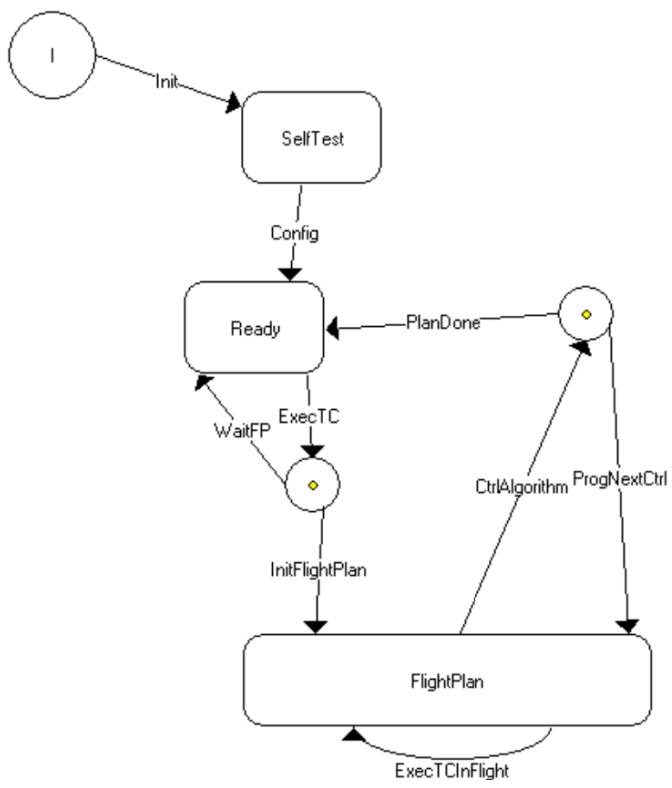
EvFlightPlanDone



5.4.1.1 Interfaces



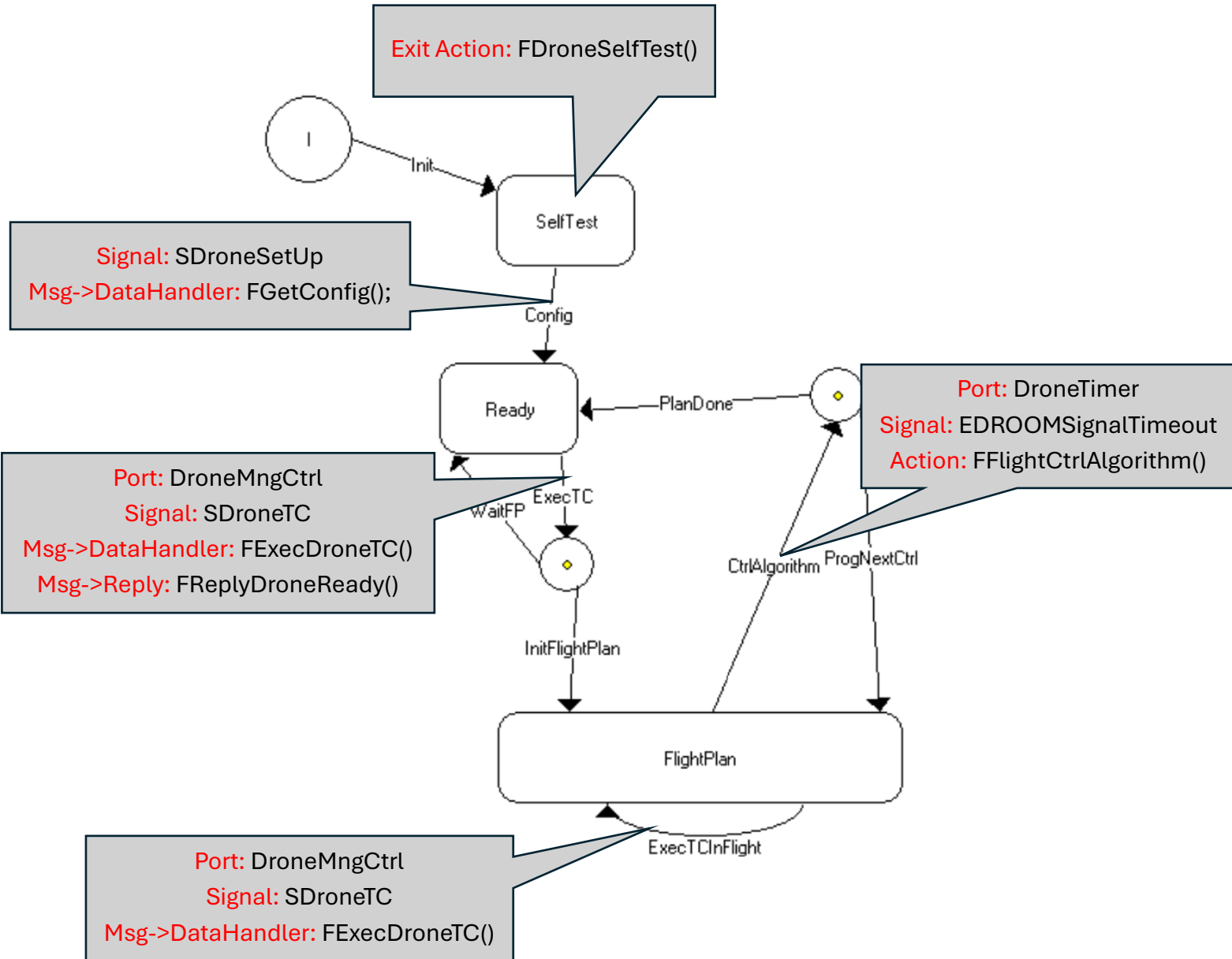
Behaviour



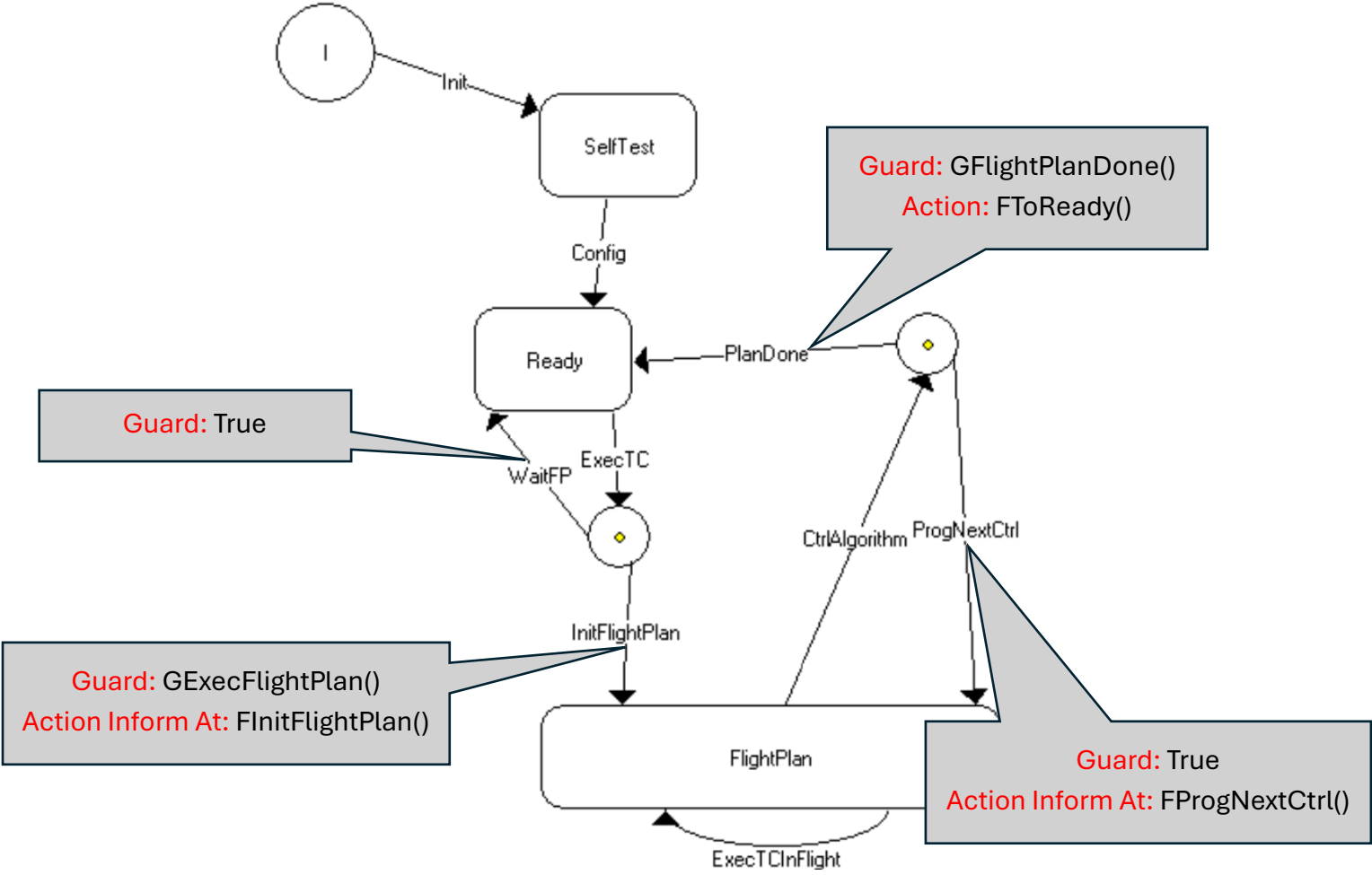
Class Variables

Variable	Class
VNextCtrl	Pr_Time

Transitions



Choice Points



Exit Action: FDroneSelfTest()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FDroneSelfTest()
{
    pus_service129_drone_selftest();
}
```

Msg->DataHandler: FGetConfig()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FGetConfig()
{
    CDDroneConfig & varSDroneSetUp = *(CDDroneConfig *)Msg->data;
    pus_service129_setup(varSDroneSetUp);
}
```

Msg->Reply: FReplyDroneReady()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FReplyDroneReady()
{
    pus_service129_drone_ready();
    Msg->reply(SDroneReady);
}
```

Msg->DataHandler: FExecDroneTC()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FExecDroneTC()
{
    CDTCHandler & varSDroneTC = *(CDTCHandler *)Msg->data;
    varSDroneTC.ExecDroneTC();
}
```

Action Inform At: FInitFlightPlan()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FInitFlightPlan()
{
    Pr_Time time;
    time.GetTime();
    time+=Pr_Time(0,100000);
    VNextCtrl=time;
    pus_service129_init_flight_plan();
    DroneTimer.InformAt( time );
}
```

Guard: GExecFlightPlan()

```
bool  CCDroneMng::EDROOM_CTX_Top_0::GExecFlightPlan()
{
    return pus_service129_flight_plan_ready();
}
```

Msg->DataHandler: FExecDroneTC()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FExecDroneTC()
{
    CDTCHandler & varSDroneTC = *(CDTCHandler *)Msg->data;
    varSDroneTC.ExecDroneTC();
}
```

Action: FFlightCtrlAlgorithm()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FFlightCtrlAlgorithm()
{
    pus_service129_flight_ctrl_algorithm();
}
```

Action Inform At: FProgNextCtrl()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FProgNextCtrl()
{
    Pr_Time time;
    VNextCtrl+= Pr_Time(0,100000); // Add X sec + Y microsec
    time=VNextCtrl;
    DroneTimer.InformAt( time );
}
```

Action: FToReady()

```
void  CCDroneMng::EDROOM_CTX_Top_0::FToReady()
{
    pus_service129_drone_ready();
}
```

Guard: GFlightPlanDone()

```
bool  CCDroneMng::EDROOM_CTX_Top_0::GFlightPlanDone()
{
    return pus_service129_flight_plan_done();
}
```